dotnet.pdf - First-Time Step-by-Step Guide

Important: the supplied code is kept as the reference. This guide explains where to put it and how to run it. I do not rewrite code unless Visual Studio reports a genuine error.

How to start in Visual Studio

1. Open Visual Studio. 2. Select Create a new project. 3. Search the exact template named for the question. 4. Give the project a simple name such as Slip1Q1. 5. Choose Create. 6. In Solution Explorer, open the file path listed below. 7. Paste the code from dotnet.pdf. 8. Save with Ctrl+S. 9. Press the green Start button or Ctrl+F5.

Template rule: Console App is for C# programs; ASP.NET Core MVC/API is modern .NET; ASP.NET Web Forms is .NET Framework. For PWA questions, use localhost or HTTPS because a Service Worker will not run from file://.

SLIP 1

Q1: Write a C# program to create methods for addition, subtraction, multiplication, and division. Use delegates to invoke these methods with suitable parameters.

Project/template: Console App
Files: Program.cs

First-time steps: Create a Console App. Open Program.cs. Copy the Q1 code from dotnet.pdf exactly. Press Ctrl+S, then Ctrl+F5. Expected output: Addition 15, Subtraction 5, Multiplication 50, Division 2. Error check: C# is case-sensitive; words shown as Using/Class/Static/Op must be typed as using/class/static/op, and curly quotation marks/dash must be normal quotes and hyphen if Visual Studio reports syntax errors.

Do not change: Copy the code from the matching question in dotnet.pdf first. Only if Visual Studio shows an error, correct the exact error and keep the same logic.

Q2: Develop a simple college website containing information about courses, faculty, and contact details. Convert it into a PWA using a manifest and service worker.

Project/template: ASP.NET Core Empty or Static Web App
Files: wwwroot/index.html, manifest.json, sw.js

First-time steps: Create an ASP.NET Core Empty project. In Solution Explorer open wwwroot. Add index.html, manifest.json, and sw.js with exactly the source-PDF code. Right-click the project and choose View in Browser, or press Ctrl+F5. Use the localhost URL, not file://. Expected: ABC College page; Application/DevTools should show the manifest and registered service worker.

Do not change: Copy the code from the matching question in dotnet.pdf first. Only if Visual Studio shows an error, correct the exact error and keep the same logic.

SLIP 2

Q1: Write a C# program to demonstrate function overloading and constructor overloading.

Project/template: Console App
Files: Program.cs

First-time steps: Create Console App, open Program.cs, paste the source-PDF code, save, and run. Expected: default and parameterized Student details, then Add results 5 and 9. If Visual Studio reports errors, correct only spelling/case errors such as void, return, public, static, Student, s1 and s2; do not redesign the program.

Do not change: Copy the code from the matching question in dotnet.pdf first. Only if Visual Studio shows an error, correct the exact error and keep the same logic.

Q2: Create an offline PWA page using a service worker.

Project/template: ASP.NET Core Empty or Static Web App
Files: wwwroot/index.html, manifest.json, sw.js

First-time steps: Create the project and add the three files under wwwroot. Paste the source-PDF code. Run once online at localhost so the cache is created. Then stop the server or select Offline in browser DevTools and reload. Expected: cached page remains available.

Do not change: Copy the code from the matching question in dotnet.pdf first. Only if Visual Studio shows an error, correct the exact error and keep the same logic.

SLIP 3

Q1: Create a BankAccount class with AccountNumber and Balance properties. Ensure the balance cannot be negative.

Project/template: Console App
Files: Program.cs

First-time steps: Create Console App and paste the source-PDF code into Program.cs. Run it. Expected: account number and 5000, followed by the negative-balance message. If the setter causes a recursive assignment error or stack overflow, the line must assign the private field balance, not the property Balance; this is an actual error fix, not a design change.

Do not change: Copy the code from the matching question in dotnet.pdf first. Only if Visual Studio shows an error, correct the exact error and keep the same logic.

Q2: Create an HTML page and manifest.json containing the app name, icons, start URL, theme color, background color, and display mode.

Project/template: ASP.NET Core Empty or Static Web App
Files: wwwroot/index.html, manifest.json, wwwroot/icons

First-time steps: Add the files under wwwroot. Create an icons folder and place 192x192 and 512x512 PNG files with the names used by the source code. Run on localhost and open DevTools -> Application -> Manifest. Expected: all manifest fields appear.

Do not change: Copy the code from the matching question in dotnet.pdf first. Only if Visual Studio shows an error, correct the exact error and keep the same logic.

SLIP 4

Q1: Develop an ASP.NET MVC calculator for arithmetic operations.

Project/template: ASP.NET Core Web App (Model-View-Controller)
Files: Controllers/HomeController.cs and Views/Home/Index.cshtml

First-time steps: Create the MVC template. Open the controller and view paths shown in the source PDF. Paste the source code in the matching files. Press Ctrl+F5, enter two numbers, submit, and compare addition, subtraction, multiplication, and division results.

Do not change: Copy the code from the matching question in dotnet.pdf first. Only if Visual Studio shows an error, correct the exact error and keep the same logic.

Q2: Create an ASP.NET Core Web API that returns student Roll Number, Name, Course, and Marks in JSON format.

Project/template: ASP.NET Core Web API
Files: Models/Student.cs and Controllers/StudentsController.cs

First-time steps: Create Web API project. Add the model and controller files from the source PDF. Run and open the shown /api/... endpoint in the browser or Swagger. Expected: a JSON student object or array.

Do not change: Copy the code from the matching question in dotnet.pdf first. Only if Visual Studio shows an error, correct the exact error and keep the same logic.

SLIP 5

Q1: Write a C# program to print Floyd’s Triangle.

Project/template: Console App
Files: Program.cs

First-time steps: Create Console App, paste the source code, run, and enter the requested number of rows. Expected for 4 rows: 1; 2 3; 4 5 6; 7 8 9 10.

Do not change: Copy the code from the matching question in dotnet.pdf first. Only if Visual Studio shows an error, correct the exact error and keep the same logic.

Q2: Create an ASP.NET Core API to perform CRUD operations on student records in a database.

Project/template: ASP.NET Core Web API
Files: Student model, DbContext, controller, appsettings.json

First-time steps: Create Web API, install the EF Core SQL Server packages named in the source PDF, add the files, check the connection string, run Add-Migration and Update-Database in Package Manager Console, then test GET/POST/PUT/DELETE in Swagger.

Do not change: Copy the code from the matching question in dotnet.pdf first. Only if Visual Studio shows an error, correct the exact error and keep the same logic.

SLIP 6

Q1: Create an ASP.NET Web Forms application using TextBox, RadioButton, DropDownList, CheckBox, Calendar, Button, and Label.

Project/template: ASP.NET Web Application (.NET Framework) -> Web Forms
Files: Default.aspx and Default.aspx.cs

First-time steps: Create the older .NET Framework Web Forms template, not ASP.NET Core. Add the controls in Default.aspx exactly as in the source PDF. Put the click event code in Default.aspx.cs. Run with IIS Express, fill controls, click Button, and read the Label.

Do not change: Copy the code from the matching question in dotnet.pdf first. Only if Visual Studio shows an error, correct the exact error and keep the same logic.

Q2: Create a PWA with a manifest and push notifications.

Project/template: ASP.NET Core Empty or Static Web App
Files: index.html, manifest.json, sw.js

First-time steps: Add the source-PDF files, run on localhost or HTTPS, allow notification permission, and test the notification event. Real push requires a subscription and server; a permission-only demo is expected to show the browser prompt.

Do not change: Copy the code from the matching question in dotnet.pdf first. Only if Visual Studio shows an error, correct the exact error and keep the same logic.

SLIP 7

Q1: Create a student registration form using RequiredFieldValidator, RangeValidator, CompareValidator, and RegularExpressionValidator.

Project/template: ASP.NET Web Application (.NET Framework) -> Web Forms
Files: Register.aspx and Register.aspx.cs

First-time steps: Create Web Forms project. Add the controls and validator markup from the source PDF to Register.aspx. Run with IIS Express. Test blank fields, an out-of-range value, mismatched values, and an invalid email.

Do not change: Copy the code from the matching question in dotnet.pdf first. Only if Visual Studio shows an error, correct the exact error and keep the same logic.

Q2: Create an e-commerce PWA with a product list and cart stored locally for offline use.

Project/template: ASP.NET Core Empty or Static Web App
Files: wwwroot/index.html, app.js, manifest.json, sw.js

First-time steps: Create the files exactly as in the source PDF. Run once online, add products, refresh, and then test offline. Expected: product list appears and cart count remains because localStorage stores the cart.

Do not change: Copy the code from the matching question in dotnet.pdf first. Only if Visual Studio shows an error, correct the exact error and keep the same logic.

SLIP 8

Q1: Create an ASP.NET Web Forms application using AdRotator and an advertisement XML file.

Project/template: ASP.NET Web Application (.NET Framework) -> Web Forms
Files: Default.aspx, Ads.xml, images folder

First-time steps: Create Web Forms project. Add AdRotator to Default.aspx. Add Ads.xml at the project root and the advertisement images under images, using the exact source-PDF paths. Run and refresh to see the selected advertisement.

Do not change: Copy the code from the matching question in dotnet.pdf first. Only if Visual Studio shows an error, correct the exact error and keep the same logic.

Q2: Convert a company website into a PWA.

Project/template: ASP.NET Core Empty or Static Web App
Files: wwwroot/index.html, manifest.json, sw.js

First-time steps: Create the website page with product, services, about and contact sections from the source PDF. Add the manifest and service worker files. Run at localhost, then verify offline loading in DevTools.

Do not change: Copy the code from the matching question in dotnet.pdf first. Only if Visual Studio shows an error, correct the exact error and keep the same logic.

SLIP 9

Q1: Write a C# program to accept ten employee names and sort them alphabetically.

Project/template: Console App
Files: Program.cs

First-time steps: Create Console App, paste the source-PDF code, run, enter ten names, and read the sorted list. Expected: names print in alphabetical order.

Do not change: Copy the code from the matching question in dotnet.pdf first. Only if Visual Studio shows an error, correct the exact error and keep the same logic.

Q2: Create an e-commerce PWA with product listing and an offline cart.

Project/template: Use the project type shown in the source PDF
Files: Use Slip 7 Q2 project

First-time steps: Use the Slip 7 Q2 files exactly as instructed in the PDF. Do not create a second design. Run once online, add products, refresh and test offline.

Do not change: Copy the code from the matching question in dotnet.pdf first. Only if Visual Studio shows an error, correct the exact error and keep the same logic.

SLIP 10

Q1: Write a C# program to demonstrate multilevel inheritance.

Project/template: Console App
Files: Program.cs

First-time steps: Create Console App, paste the source code, run, and observe methods from the base, derived, and final class. Expected: messages from all three levels.

Do not change: Copy the code from the matching question in dotnet.pdf first. Only if Visual Studio shows an error, correct the exact error and keep the same logic.

Q2: Create a PWA that caches HTML, CSS, JavaScript, and images for offline use.

Project/template: ASP.NET Core Empty or Static Web App
Files: wwwroot/index.html, style.css, app.js, images, sw.js

First-time steps: Add every file under wwwroot using the source-PDF names. Run once online, reload, switch DevTools to Offline, and reload. Expected: cached page, style, script and image continue to appear.

Do not change: Copy the code from the matching question in dotnet.pdf first. Only if Visual Studio shows an error, correct the exact error and keep the same logic.

SLIP 11

Q1: Write a C# program to demonstrate multiple inheritance using interfaces.

Project/template: Console App
Files: Program.cs

First-time steps: Create Console App, paste the source code, run, and observe both interface methods. Viva point: C# uses multiple interfaces rather than multiple base classes.

Do not change: Copy the code from the matching question in dotnet.pdf first. Only if Visual Studio shows an error, correct the exact error and keep the same logic.

Q2: Write a C# program to demonstrate constructor overloading.

Project/template: Console App
Files: Program.cs

First-time steps: Create Console App, paste the source code, create objects using each constructor, run, and compare the initialized values.

Do not change: Copy the code from the matching question in dotnet.pdf first. Only if Visual Studio shows an error, correct the exact error and keep the same logic.

SLIP 12

Q1: Write a program to demonstrate the use of an interface/contract.

Project/template: Console App
Files: Program.cs

First-time steps: Create Console App, paste the interface and implementing class from the source PDF, run, and call the implemented method.

Do not change: Copy the code from the matching question in dotnet.pdf first. Only if Visual Studio shows an error, correct the exact error and keep the same logic.

Q2: Write a program to demonstrate exception handling using try, catch, finally, custom exception, and multiple catch blocks.

Project/template: Console App
Files: Program.cs

First-time steps: Create Console App, paste the source code, run once with valid input, once with invalid format, and once with the custom-error input. Observe which catch block runs and that finally executes.

Do not change: Copy the code from the matching question in dotnet.pdf first. Only if Visual Studio shows an error, correct the exact error and keep the same logic.

SLIP 13

Q1: Write an ASP.NET MVC program to perform addition, subtraction, multiplication, and division.

Project/template: ASP.NET Core Web App (Model-View-Controller)
Files: Controller and View files named in the source PDF

First-time steps: Create MVC project, paste the source code into its matching controller/view files, run, enter two numbers and submit. Expected: four results on the page.

Do not change: Copy the code from the matching question in dotnet.pdf first. Only if Visual Studio shows an error, correct the exact error and keep the same logic.

Q2: Develop a PWA that works offline.

Project/template: ASP.NET Core Empty or Static Web App
Files: wwwroot/index.html, manifest.json, sw.js

First-time steps: Add the source-PDF files. Run once online on localhost, then use DevTools Offline and reload. Expected: cached page is returned.

Do not change: Copy the code from the matching question in dotnet.pdf first. Only if Visual Studio shows an error, correct the exact error and keep the same logic.

SLIP 14

Q1: Write a C# program to print Floyd’s Triangle.

Project/template: Console App
Files: Program.cs

First-time steps: Follow Slip 5 Q1: create Console App, paste the source code, run, enter rows, and compare the triangular output.

Do not change: Copy the code from the matching question in dotnet.pdf first. Only if Visual Studio shows an error, correct the exact error and keep the same logic.

Q2: Write a C# program to accept ten city names and display them in alphabetical order.

Project/template: Console App
Files: Program.cs

First-time steps: Create Console App, paste the source code, enter ten city names, and run. Expected: names print A-Z.

Do not change: Copy the code from the matching question in dotnet.pdf first. Only if Visual Studio shows an error, correct the exact error and keep the same logic.

Common genuine errors

Red underline under quotes or minus: PDF typography may display curly quotes or a long dash. In a C# or JSON file, replace only those characters with normal straight quotes (" ") and a normal minus (-).

Capitalization error: C# keywords and variable names are case-sensitive. If the source shows Using, Class, Static, If, Return, Navigator or Caches with an error, use the spelling Visual Studio suggests; this is a compile-error correction, not a logic change.

Web Forms does not appear: choose ASP.NET Web Application (.NET Framework), install the ASP.NET and web development workload, and select Web Forms.

Database connection error: check SQL Server/LocalDB is installed and running, then check the connection string before changing code.

PWA does not work offline: run on localhost/HTTPS, load once online, check DevTools -> Application -> Service Workers, then switch Network to Offline.

Final check: build with Ctrl+Shift+B. If Build succeeded appears, run the project and compare the output with the expected result in the slip.

SLIP 1

Q1. Write a C# program to create methods for addition, subtraction, multiplication, and division. Use delegates to invoke these methods with suitable parameters.

```
Using System;

Delegate double Operation(double a, double b);

Class Program
{
   Static double Add(double a, double b) => a + b;
   Static double Sub(double a, double b) => a – b;
   Static double Mul(double a, double b) => a * b;
   Static double Div(double a, double b) => a / b;

   Static void Main()
   {
      Operation op;

      Op = Add;
      Console.WriteLine(“Addition: “ + op(10, 5));

      Op = Sub;
      Console.WriteLine(“Subtraction: “ + op(10, 5));

      Op = Mul;
      Console.WriteLine(“Multiplication: “ + op(10, 5));

      Op = Div;
      Console.WriteLine(“Division: “ + op(10, 5));
   }
}
```

Q2. Develop a simple college website containing information about courses, faculty, and contact details. Convert it into a PWA using a manifest and service worker.

Index.html

```
<!DOCTYPE html>
<html>
<head>
   <title>College Website</title>
   <link rel=”manifest” href=”manifest.json”>
</head>
<body>
   <h1>ABC College</h1>
   <h2>Courses</h2>
   <p>BSc IT, BCom, BA</p>

   <h2>Faculty</h2>
   <p>Experienced teaching staff</p>

   <h2>Contact</h2>
   <p>Email: college@example.com</p>

   <script>
      If (“serviceWorker” in navigator)
         Navigator.serviceWorker.register(“sw.js”);
   </script>
</body>
</html>
```

Manifest.json

```
{
  “name”: “College Website”,
  “short_name”: “College”,
  “start_url”: “index.html”,
  “display”: “standalone”,
```

```
  “background_color”: “#ffffff”,
  “theme_color”: “#1976d2”
}
```

Sw.js

```
Const CACHE = “college-v1”;

Self.addEventListener(“install”, event => {
   Event.waitUntil(
      Caches.open(CACHE).then(cache =>
         Cache.addAll([“index.html”, “manifest.json”])
      )
   );
});

Self.addEventListener(“fetch”, event => {
   Event.respondWith(
      Caches.match(event.request).then(
         Response => response || fetch(event.request)
      )
   );
});
```

SLIP 2

Q1. Write a C# program to demonstrate function overloading and constructor overloading.

```
Using System;

Class Student
{
   String name;
   Int age;

   Public Student()
   {
      Name = “Unknown”;
      Age = 0;
   }

   Public Student(string n, int a)
   {
      Name = n;
      Age = a;
   }

   Void Display()
   {
      Console.WriteLine(name + “ “ + age);
   }

   Int Add(int a, int b)
   {
      Return a + b;
   }

   Int Add(int a, int b, int c)
   {
      Return a + b + c;
   }

   Static void Main()
   {
      Student s1 = new Student();
      Student s2 = new Student(“Student”, 20);

      S1.Display();
      S2.Display();
```

```
        Console.WriteLine(s2.Add(2, 3));
        Console.WriteLine(s2.Add(2, 3, 4));
    }
}
```

Q2. Create an offline PWA page using a service worker.

Index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Offline PWA</title>
    <link rel=”manifest” href=”manifest.json”>
</head>
<body>
    <h1>Welcome to Offline PWA</h1>
    <p>This page can work offline.</p>

    <script>
        Navigator.serviceWorker.register(“sw.js”);
    </script>
</body>
</html>
```

Manifest.json

```
{
  “name”: “Offline PWA”,
  “short_name”: “Offline”,
  “start_url”: “index.html”,
  “display”: “standalone”,
  “background_color”: “#ffffff”,
  “theme_color”: “#0000ff”
}
```

Sw.js

```
Self.addEventListener(“install”, event => {
    Event.waitUntil(
        Caches.open(“offline-v1”).then(cache =>
            Cache.addAll([“index.html”, “manifest.json”])
        )
    );
});

Self.addEventListener(“fetch”, event => {
    Event.respondWith(
        Caches.match(event.request).then(
            Response => response || fetch(event.request)
        )
    );
});
```

SLIP 3

Q1. Create a BankAccount class with AccountNumber and Balance properties. Ensure the balance cannot be negative.

```
Using System;

Class BankAccount
{
    Public string AccountNumber { get; set; }

    Private double balance;

    Public double Balance
    {
```

```
        Get { return balance; }
        Set
        {
            If (value >= 0)
                Balance = value;
            Else
                Console.WriteLine("Balance cannot be negative.");
        }
    }

    Static void Main()
    {
        BankAccount b = new BankAccount();

        b.AccountNumber = "12345";
        b.Balance = 5000;

        Console.WriteLine(b.AccountNumber);
        Console.WriteLine(b.Balance);

        b.Balance = -100;
    }
}
```

Q2. Create an HTML page and manifest.json containing the app name, icons, start URL, theme color, background color, and display mode.

Index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>My PWA</title>
    <link rel="manifest" href="manifest.json">
</head>
<body>
    <h1>My Progressive Web App</h1>
</body>
</html>
```

Manifest.json

```
{
  "name": "My Progressive Web App",
  "short_name": "MyPWA",
  "start_url": "index.html",
  "display": "standalone",
  "theme_color": "#1976d2",
  "background_color": "#ffffff",
  "icons": [
    {
      "src": "icon-192.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "icon-512.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

Place the two icon files in the same folder.

SLIP 4

Q1. Develop an ASP.NET MVC calculator for arithmetic operations.

CalculatorController.cs

```
Using Microsoft.AspNetCore.Mvc;

Public class CalculatorController : Controller
{
   Public IActionResult Index()
   {
      Return View();
   }

   [HttpPost]
   Public IActionResult Index(double a, double b, string op)
   {
      Double result = 0;

      Switch (op)
      {
         Case “+”: result = a + b; break;
         Case “-“: result = a – b; break;
         Case “*”: result = a * b; break;
         Case “/”:
            If (b == 0)
               Return Content(“Cannot divide by zero”);
            Result = a / b;
            Break;
      }

      ViewBag.Result = result;
      Return View();
   }
}
```

Views/Calculator/Index.cshtml

```
<form method=”post”>
   <input type=”number” step=”any” name=”a” required>
   <input type=”number” step=”any” name=”b” required>

   <select name=”op”>
      <option>+</option>
      <option>-</option>
      <option>*</option>
      <option>/</option>
   </select>

   <button type=”submit”>Calculate</button>
</form>

<h3>Result: @ViewBag.Result</h3>
```

Q2. Create an ASP.NET Core Web API that returns student Roll Number, Name, Course, and Marks in JSON format.

Program.cs

```
Var builder = WebApplication.CreateBuilder(args);
Var app = builder.Build();

App.MapGet(“/student”, () => new
{
   RollNumber = 101,
   Name = “Student”,
   Course = “BSc IT”,
   Marks = 85
});

App.Run();
```

Run the application and open “/student” to view the JSON response.

SLIP 5

Q1. Write a C# program to print Floyd's Triangle.

```
Using System;

Class Program
{
   Static void Main()
   {
      Int n = 5, num = 1;

      For (int I = 1; I <= n; i++)
      {
         For (int j = 1; j <= I; j++)
            Console.Write(num++ + " ");

         Console.WriteLine();
      }
   }
}
```

Q2. Create an ASP.NET Core API to perform CRUD operations on student records in a database.

Program.cs — simple in-memory CRUD API

```
Var builder = WebApplication.CreateBuilder(args);
Var app = builder.Build();

Var students = new List<object>();

App.MapGet("/students", () => students);

App.MapPost("/students", (object student) =>
{
   Students.Add(student);
   Return Results.Ok("Student added");
});

App.MapPut("/students/{id}", (int id, object student) =>
{
   If (id < 0 || id >= students.Count)
      Return Results.NotFound();

   Students[id] = student;
   Return Results.Ok("Student updated");
});

App.MapDelete("/students/{id}", (int id) =>
{
   If (id < 0 || id >= students.Count)
      Return Results.NotFound();

   Students.RemoveAt(id);
   Return Results.Ok("Student deleted");
});

App.Run();
```

Note: This is a short demonstration of CRUD operations using an in-memory list, not a database-backed implementation. A real database version requires database configuration and a student model.

SLIP 6

Q1. Create an ASP.NET Web Forms application using TextBox, RadioButton, DropDownList, CheckBox, Calendar, Button, and Label.

Default.aspx

```
<form id="form1" runat="server">
```

```
    Name:
    <asp:TextBox ID=”txtName” runat=”server” />
    <br />

    Gender:
    <asp:RadioButton ID=”rbMale” Text=”Male”
        GroupName=”Gender” runat=”server” />
    <asp:RadioButton ID=”rbFemale” Text=”Female”
        GroupName=”Gender” runat=”server” />
    <br />

    Course:
    <asp:DropDownList ID=”ddlCourse” runat=”server”>
        <asp:ListItem>BSc IT</asp:ListItem>
        <asp:ListItem>BCA</asp:ListItem>
    </asp:DropDownList>
    <br />

    <asp:CheckBox ID=”chkAgree” Text=”I Agree”
        Runat=”server” />
    <br />

    <asp:Calendar ID=”calDate” runat=”server” />
    <br />

    <asp:Button ID=”btnSubmit” Text=”Submit”
        OnClick=”btnSubmit_Click” runat=”server” />
    <br />

    <asp:Label ID=”lblResult” runat=”server” />
</form>
```

Default.aspx.cs

```
Protected void btnSubmit_Click(object sender, EventArgs e)
{
    String gender = rbMale.Checked ? “Male” : “Female”;

    lblResult.Text = “Name: “ + txtName.Text +
        “<br>Gender: “ + gender +
        “<br>Course: “ + ddlCourse.SelectedValue +
        “<br>Agreed: “ + chkAgree.Checked +
        “<br>Date: “ + calDate.SelectedDate.ToShortDateString();
}
```

Q2. Create a PWA with a manifest and push notifications.

Index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Notification PWA</title>
    <link rel=”manifest” href=”manifest.json”>
</head>
<body>
    <h1>Notification PWA</h1>
    <button onclick=”notify()”>Show Notification</button>

    <script>
        Navigator.serviceWorker.register(“sw.js”);

        Function notify() {
            Notification.requestPermission().then(permission => {
                If (permission === “granted”) {
                    Navigator.serviceWorker.ready.then(reg => {
                        Reg.showNotification(“Welcome!”, {
                            Body: “New notification received.”
                        });
                    });
```

```
            }
        });
    }
  </script>
</body>
</html>
```

Manifest.json

```
{
 “name”: “Notification PWA”,
 “short_name”: “Notify”,
 “start_url”: “index.html”,
 “display”: “standalone”,
 “theme_color”: “#1976d2”,
 “background_color”: “#ffffff”
}
```

Sw.js

```
Self.addEventListener(“install”, event => {
  Event.waitUntil(
    Caches.open(“notify-v1”).then(cache =>
      Cache.addAll([“index.html”, “manifest.json”])
    )
  );
});

Self.addEventListener(“fetch”, event => {
  Event.respondWith(
    Caches.match(event.request).then(
      Response => response || fetch(event.request)
    )
  );
});
```

Note: This demonstrates a local notification. Real push notifications require push-service configuration and subscription handling.

SLIP 7

Q1. Create a student registration form using RequiredFieldValidator, RangeValidator, CompareValidator, and RegularExpressionValidator.

Default.aspx

```
<form runat=”server”>
  Name:
  <asp:TextBox ID=”txtName” runat=”server” />
  <asp:RequiredFieldValidator
    ControlToValidate=”txtName”
    ErrorMessage=”Name required”
    Runat=”server” />
  <br />

  Age:
  <asp:TextBox ID=”txtAge” runat=”server” />
  <asp:RangeValidator
    ControlToValidate=”txtAge”
    MinimumValue=”18” MaximumValue=”60”
    Type=”Integer”
    ErrorMessage=”Age must be 18 to 60”
    Runat=”server” />
  <br />

  Password:
  <asp:TextBox ID=”txtPass” TextMode=”Password”
    Runat=”server” />
  <br />

  Confirm Password:
```

```
    <asp:TextBox ID=”txtConfirm” TextMode=”Password”
      Runat=”server” />
    <asp:CompareValidator
      ControlToValidate=”txtConfirm”
      ControlToCompare=”txtPass”
      ErrorMessage=”Passwords do not match”
      Runat=”server” />
    <br />

    Email:
    <asp:TextBox ID=”txtEmail” runat=”server” />
    <asp:RegularExpressionValidator
      ControlToValidate=”txtEmail”
      ValidationExpression=”\w+([.-]?\w+)*@\w+([.-]?\w+)*(\.\w{2,3})+”
      ErrorMessage=”Invalid email”
      Runat=”server” />
    <br />

    <asp:Button Text=”Register” runat=”server” />
    <asp:ValidationSummary runat=”server” />
</form>
```

Q2. Create an e-commerce PWA with a product list and cart stored locally for offline use.

Index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Shopping PWA</title>
    <link rel=”manifest” href=”manifest.json”>
</head>
<body>
    <h1>Products</h1>

    <p>T-Shirt - ₹500
      <button onclick=”add(‘T-Shirt’)”>Add</button>
    </p>

    <p>Shoes - ₹1000
      <button onclick=”add(‘Shoes’)”>Add</button>
    </p>

    <h2>Cart</h2>
    <ul id=”cart”></ul>

    <script>
      Let items = JSON.parse(localStorage.cart || “[]”);

      Function show() {
        Cart.innerHTML = items.map(item =>
          `<li>${item}</li>`
        ).join(“”);
      }

      Function add(item) {
        Items.push(item);
        localStorage.cart = JSON.stringify(items);
        show();
      }

      Navigator.serviceWorker.register(“sw.js”);
      Show();
    </script>
</body>
</html>
```

Manifest.json

```
{
```

```
  “name”: “Shopping PWA”,
  “short_name”: “Shopping”,
  “start_url”: “index.html”,
  “display”: “standalone”
}
```

Sw.js

```
Self.addEventListener(“install”, event => {
   Event.waitUntil(
      Caches.open(“shop-v1”).then(cache =>
         Cache.addAll([“index.html”, “manifest.json”])
      )
   );
});

Self.addEventListener(“fetch”, event => {
   Event.respondWith(
      Caches.match(event.request).then(
         Response => response || fetch(event.request)
      )
   );
});
```

SLIP 8

Q1. Create an ASP.NET Web Forms application using AdRotator and an advertisement XML file.

Default.aspx

```
<form runat=”server”>
   <asp:AdRotator ID=”AdRotator1”
      AdvertisementFile=”Ads.xml”
      Runat=”server” />
</form>
```

Ads.xml

```
<Advertisements>
   <Ad>
      <ImageUrl>ad1.jpg</ImageUrl>
      <NavigateUrl>https://example.com</NavigateUrl>
      <AlternateText>Advertisement One</AlternateText>
      <Impressions>50</Impressions>
      <Keyword>Education</Keyword>
   </Ad>
   <Ad>
      <ImageUrl>ad2.jpg</ImageUrl>
      <NavigateUrl>https://example.org</NavigateUrl>
      <AlternateText>Advertisement Two</AlternateText>
      <Impressions>50</Impressions>
      <Keyword>Shopping</Keyword>
   </Ad>
</Advertisements>
```

Place “Ads.xml”, “ad1.jpg”, and “ad2.jpg” in the correct application folder.

Q2. Convert a company website into a PWA.

Index.html

```
<!DOCTYPE html>
<html>
<head>
   <title>Company Website</title>
   <link rel=”manifest” href=”manifest.json”>
</head>
<body>
   <h1>ABC Company</h1>
```

```
    <p>Welcome to our company website.</p>

    <script>
        Navigator.serviceWorker.register("sw.js");
    </script>
</body>
</html>
```

Manifest.json

```
{
  "name": "Company Website",
  "short_name": "Company",
  "start_url": "index.html",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#1976d2"
}
```

Sw.js

```
Self.addEventListener("install", event => {
    Event.waitUntil(
        Caches.open("company-v1").then(cache =>
            Cache.addAll(["index.html", "manifest.json"])
        )
    );
});

Self.addEventListener("fetch", event => {
    Event.respondWith(
        Caches.match(event.request).then(
            Response => response || fetch(event.request)
        )
    );
});
```

SLIP 9

Q1. Write a C# program to accept ten employee names and sort them alphabetically.

```
Using System;

Class Program
{
    Static void Main()
    {
        String[] names = new string[10];

        For (int I = 0; I < 10; i++)
        {
            Console.Write("Enter employee name: ");
            Names[i] = Console.ReadLine();
        }

        Array.Sort(names);

        Console.WriteLine("Sorted Names:");

        Foreach (string name in names)
            Console.WriteLine(name);
    }
}
```

Q2. Create an e-commerce PWA with product listing and an offline cart.

Use the Slip 7, Q2 code. It already includes a product list, a cart saved in localStorage, a manifest, and a service worker.

SLIP 10

Q1. Write a C# program to demonstrate multilevel inheritance.

```
Using System;

Class Animal
{
   Public void Eat()
   {
      Console.WriteLine(“Animal eats”);
   }
}

Class Dog : Animal
{
   Public void Bark()
   {
      Console.WriteLine(“Dog barks”);
   }
}

Class Puppy : Dog
{
   Public void Play()
   {
      Console.WriteLine(“Puppy plays”);
   }
}

Class Program
{
   Static void Main()
   {
      Puppy p = new Puppy();
      p.Eat();
      p.Bark();
      p.Play();
   }
}
```

Q2. Create a PWA that caches HTML, CSS, JavaScript, and images for offline use.

Index.html

```
<!DOCTYPE html>
<html>
<head>
   <title>Offline Website</title>
   <link rel=”stylesheet” href=”style.css”>
   <link rel=”manifest” href=”manifest.json”>
</head>
<body>
   <h1>Offline Website</h1>
   <img src=”image.png” width=”100”>

   <script src=”app.js”></script>
   <script>
      Navigator.serviceWorker.register(“sw.js”);
   </script>
</body>
</html>
```

Style.css

```
Body {
   Font-family: Arial;
   Text-align: center;
}
```

App.js

```
Console.log(“Website loaded”);
```

Manifest.json

```
{
  “name”: “Offline Website”,
  “short_name”: “Offline”,
  “start_url”: “index.html”,
  “display”: “standalone”
}
```

Sw.js

```
Const FILES = [
   “index.html”, “style.css”, “app.js”,
   “manifest.json”, “image.png”
];

Self.addEventListener(“install”, event => {
   Event.waitUntil(
      Caches.open(“offline-v1”).then(cache =>
         Cache.addAll(FILES)
      )
   );
});

Self.addEventListener(“fetch”, event => {
   Event.respondWith(
      Caches.match(event.request).then(
         Response => response || fetch(event.request)
      )
   );
});
```

Keep all five files and the image in the correct folder. If any listed file is missing, caching can fail.

SLIP 11

Q1. Write a C# program to demonstrate multiple inheritance using interfaces.

```
Using System;

Interface IPrintable
{
   Void Print();
}

Interface IShowable
{
   Void Show();
}

Class Demo : IPrintable, IShowable
{
   Public void Print()
   {
      Console.WriteLine(“Printing”);
   }

   Public void Show()
   {
      Console.WriteLine(“Showing”);
   }

   Static void Main()
   {
      Demo d = new Demo();
```

```
        d.Print();
        d.Show();
    }
}
```

Q2. Write a C# program to demonstrate constructor overloading.

```
Using System;

Class Student
{
    String name;
    Int age;

    Public Student()
    {
        Name = “Unknown”;
        Age = 0;
    }

    Public Student(string n, int a)
    {
        Name = n;
        Age = a;
    }

    Void Display()
    {
        Console.WriteLine(name + “ “ + age);
    }

    Static void Main()
    {
        Student s1 = new Student();
        Student s2 = new Student(“Student”, 20);

        S1.Display();
        S2.Display();
    }
}
```

SLIP 12

Q1. Write a program to demonstrate the use of an interface/contract.

```
Using System;

Interface IShape
{
    Void CalculateArea();
}

Class Circle : IShape
{
    Public void CalculateArea()
    {
        Double r = 5;
        Double area = 3.14 * r * r;
        Console.WriteLine(“Area of Circle = “ + area);
    }
}

Class Program
{
    Static void Main()
    {
        Circle c = new Circle();
        c.CalculateArea();
    }
}
```

Q2. Write a program to demonstrate exception handling using try, catch, finally, custom exception, and multiple catch blocks.

```
Using System;

Class MyException : Exception
{
   Public MyException(string message) : base(message)
   {
   }
}

Class Program
{
   Static void Main()
   {
      Try
      {
         Console.Write(“Enter a number: “);
         Int a = Convert.ToInt32(Console.ReadLine());

         Console.Write(“Enter another number: “);
         Int b = Convert.ToInt32(Console.ReadLine());

         If (b == 0)
         {
            Throw new MyException(“Division by zero is not allowed.”);
         }

         Console.WriteLine(“Result = “ + (a / b));
      }
      Catch (MyException e)
      {
         Console.WriteLine(“Custom Exception: “ + e.Message);
      }
      Catch (FormatException)
      {
         Console.WriteLine(“Please enter valid numbers.”);
      }
      Catch (Exception e)
      {
         Console.WriteLine(“Error: “ + e.Message);
      }
      Finally
      {
         Console.WriteLine(“Finally block executed.”);
      }
   }
}
```

SLIP 13

Q1. Write an ASP.NET MVC program to perform addition, subtraction, multiplication, and division.

```
Using System;
Using System.Web.Mvc;

Public class CalculatorController : Controller
{
   Public ActionResult Index()
   {
      Return View();
   }

   [HttpPost]
   Public ActionResult Index(int a, int b, string operation)
   {
      Double result = 0;
```

```
        Switch (operation)
        {
            Case “Add”:
                Result = a + b;
                Break;

            Case “Subtract”:
                Result = a – b;
                Break;

            Case “Multiply”:
                Result = a * b;
                Break;

            Case “Divide”:
                If (b != 0)
                    Result = (double)a / b;
                Else
                    ViewBag.Result = “Cannot divide by zero.”;
                Break;
        }

        If (b != 0 || operation != “Divide”)
            ViewBag.Result = result;

        Return View();
    }
}
```

Q2. Develop a PWA that works offline.

Index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Offline PWA</title>
    <link rel=”manifest” href=”manifest.json”>
</head>
<body>
    <h1>Welcome to My PWA</h1>
    <p>This application works offline.</p>

    <script>
        If (“serviceWorker” in navigator) {
            Navigator.serviceWorker.register(“service-worker.js”);
        }
    </script>
</body>
</html>
```

Manifest.json

```
{
    “name”: “Offline PWA”,
    “short_name”: “PWA”,
    “start_url”: “index.html”,
    “display”: “standalone”,
    “background_color”: “#ffffff”,
    “theme_color”: “#0000ff”
}
```

Service-worker.js

```
Const CACHE_NAME = “pwa-cache”;

Const FILES = [
    “index.html”,
    “manifest.json”
```

```
];

Self.addEventListener("install", event => {
  Event.waitUntil(
    Caches.open(CACHE_NAME).then(cache => {
      Return cache.addAll(FILES);
    })
  );
});

Self.addEventListener("fetch", event => {
  Event.respondWith(
    Caches.match(event.request).then(response => {
      Return response || fetch(event.request);
    })
  );
});
```

SLIP 14

Q1. Write a C# program to print Floyd's Triangle.

```
Using System;

Class Program
{
  Static void Main()
  {
    Int n = 5;
    Int num = 1;

    For (int I = 1; I <= n; i++)
    {
      For (int j = 1; j <= I; j++)
      {
        Console.Write(num + " ");
        Num++;
      }

      Console.WriteLine();
    }
  }
}
```

Q2. Write a C# program to accept ten city names and display them in alphabetical order.

```
Using System;

Class Program
{
  Static void Main()
  {
    String[] cities = new string[10];

    Console.WriteLine("Enter 10 city names:");

    For (int I = 0; I < 10; i++)
    {
      Cities[i] = Console.ReadLine();
    }

    Array.Sort(cities);

    Console.WriteLine("Cities in alphabetical order:");

    For (int I = 0; I < 10; i++)
    {
      Console.WriteLine(cities[i]);
```

```
    }
  }
}
```